SENIOR ENGINEER HANDBOOK

Advanced Java & Spring Boot

Production-Grade Backend Engineering

A comprehensive deep-dive into JVM internals, memory management, Spring ecosystem, Hibernate, Security, and real-world production patterns. From source-level understanding to battle-tested implementation.

JVM Internals · Spring Boot 3.x · Hibernate ORM · Spring Security + JWT · GC Tuning · HikariCP · Redis · Microservices

Table of Contents

Complete coverage from JVM internals to production deployment

1 **JVM Internals — Deep Dive** 4

Class Loading, Bytecode, Execution Engine, JIT

2 **Memory Management** 12

Heap Structure, GC Algorithms, G1 & ZGC Tuning

3 **Spring IoC & Dependency Injection** 20

Container Internals, Bean Lifecycle, Scopes

4 **Spring MVC — Request Pipeline** 28

DispatcherServlet, Filters, Exception Handling

5 **Spring Boot Internals** 36

Auto-Configuration, Starters, Actuator

6 **HikariCP — Connection Pooling** 44

Pool Sizing, Monitoring, Production Config

7 **JPA & Hibernate Internals** 52

Session, Dirty Checking, Caching, N+1 Fix

8 **Spring Security + JWT** 60

Authentication, Authorization, Filter Chain

9 **Redis Caching & Rate Limiting** 68

Cache-Aside, TTL, Distributed Rate Limiting

10 **Project — URL Shortener** 76

Production-Grade Architecture & Full Code

CHAPTER 01

JVM Internals

Class loading, bytecode structure, the execution engine, and JIT compilation — understanding what happens from `.java` to machine code.

1.1 What Is the JVM?

The Java Virtual Machine (JVM) is an abstract computing machine that enables a computer to run Java programs. It provides a runtime environment in which Java bytecode can be executed. The JVM is specification-driven — many implementations exist (HotSpot, GraalVM, OpenJ9, Azul Zing), but all share the same class file format and bytecode instruction set.

The JVM architecture has three primary subsystems: the **Class Loader Subsystem**, the **Runtime Data Areas** (memory model), and the **Execution Engine**. Each plays a critical role in how Java code transforms from human-readable source into native machine execution.

📂 Class Loader Subsystem

Loads .class files, verifies bytecode, prepares memory, and resolves symbolic references.

🧠 Runtime Data Areas

Method Area, Heap, Java Stacks, PC Registers, Native Method Stacks — each with distinct scope and lifetime.

⚙️ Execution Engine

Interprets bytecode or compiles to native code via JIT. Also manages garbage collection.

🔗 Native Interface (JNI)

Bridges Java and native C/C++ libraries. Used for OS-level operations and hardware access.

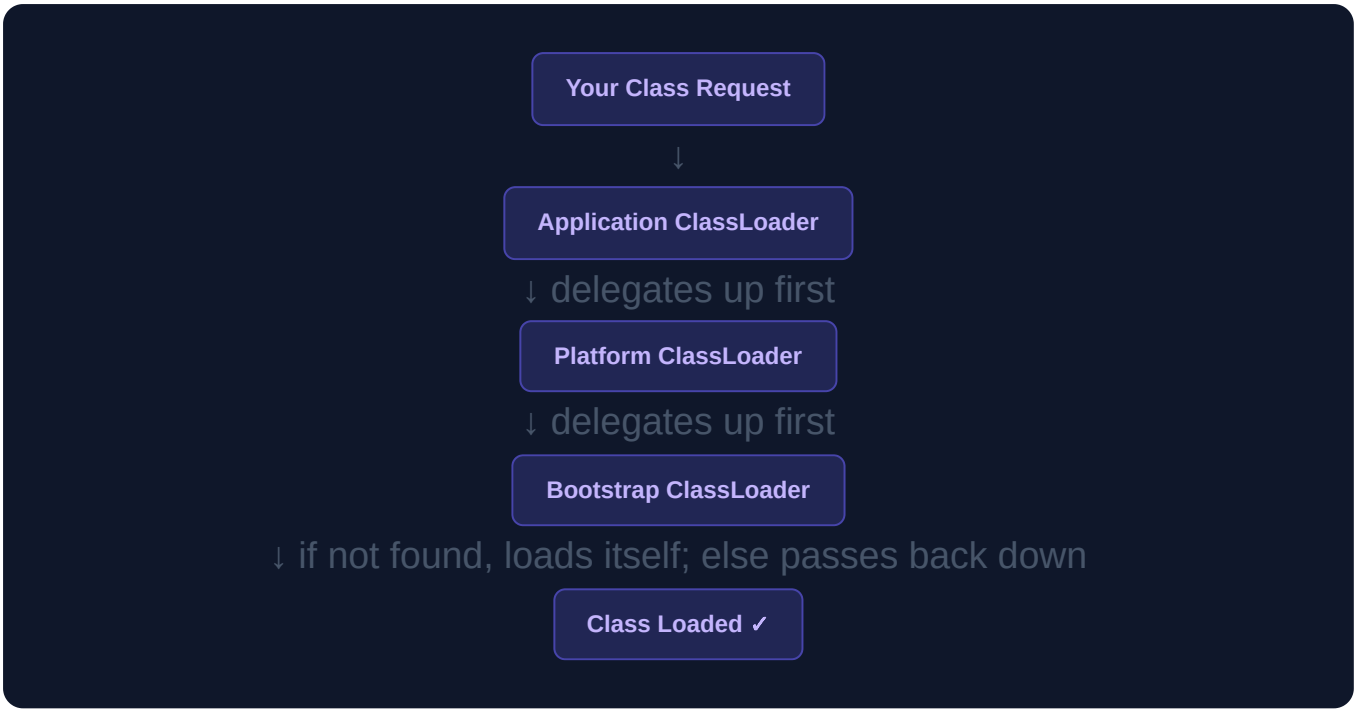
1.2 Class Loading Process — Deep Dive

Class loading is a multi-phase process that transforms a binary `.class` file stored on disk into a live `Class` object in the JVM's memory. It happens lazily — classes are loaded on first use, not at startup.

Phase 1: Loading

The JVM reads the binary data of a `.class` file (from filesystem, JAR, network, or even dynamically generated) and creates a `java.lang.Class` object in the Method Area (or Metaspace in Java 8+). The fully-qualified class name, its superclass, and all implemented interfaces are recorded.

The **Bootstrap ClassLoader** loads core JDK classes (`java.lang.*`, `java.util.*`). The **Extension/Platform ClassLoader** loads JDK extensions. The **Application ClassLoader** loads your application's classpath. This forms the **Parent Delegation Model**.



> 📘 **WHY PARENT DELEGATION?**
>
> It prevents malicious code from overriding core JDK classes. If you create a `java.lang.String` class in your project, the Bootstrap ClassLoader always wins — your version is never loaded.

Phase 2: Linking

Linking is itself a three-step process: **Verification**, **Preparation**, and **Resolution**.

1. **VERIFICATION**

   The bytecode verifier checks that the class file is structurally correct, that all opcodes are valid, that the stack doesn't overflow or underflow, that type rules are respected, and that there are no illegal jumps. This is a security and correctness gate — it prevents the JVM from crashing due to malformed class files.

2. **PREPARATION**

   Memory is allocated for the class's static fields, and they are set to their *default* values (not the values you assigned in code). So `static int x = 10` becomes `static int x = 0` at preparation time. Your assigned value comes during initialization.

3. **RESOLUTION**

   Symbolic references in the constant pool (e.g., `UserService` or `java.util.List`) are replaced with direct memory references. This may trigger the loading of additional classes recursively.

Phase 3: Initialization

Static initializers and static field assignments run here. The JVM guarantees this is thread-safe. The class initializer (`<clinit>`) method is invoked. This is when `static int x = 10` finally becomes 10.

JAVA ClassLoading.java

```java
public class A {
    static int x = 10;  // Preparation: x=0 | Initialization: x=10

    static {
        System.out.println("A's static initializer runs during Initialization");
    }
}

// Bytecode produced (javap -c A.class):
// 0: bipush        10    ← push int 10 onto operand stack
// 2: putstatic     #2    ← store into static field x
// 5: return
```

1.3 Runtime Data Areas (Memory Model)

The JVM organizes memory into several distinct runtime data areas, each with different lifecycle, thread ownership, and garbage collection behavior.

Area	Per-Thread or Shared	Description	GC Managed?
Method Area / Metaspace	Shared	Class metadata, static variables, constant pool, compiled code	Partial (class unloading)
Heap	Shared	All object instances and arrays	Yes (GC)
Java Stack	Per-Thread	Stack frames: local vars, operand stack, return address	No
PC Register	Per-Thread	Points to next instruction to execute	No
Native Method Stack	Per-Thread	Supports native (C/C++) method execution via JNI	No

1.4 Execution Engine — Interpreter vs JIT

The Execution Engine reads bytecode and executes it. Modern JVMs use a two-tier strategy: first interpret, then JIT-compile hot methods.

Interpreter

The interpreter reads each bytecode opcode and executes it directly. It's fast to start but slow at runtime because it re-executes the same opcode-to-native translation on every call. Suitable for code paths called

only a few times.

JIT Compiler (Just-In-Time)

When the JVM detects a method is "hot" (called frequently — default threshold: ~10,000 invocations in HotSpot), it compiles the entire method to native machine code. Future calls bypass bytecode interpretation entirely. HotSpot has two compilers: **C1** (client compiler — fast compilation, minimal optimization) and **C2** (server compiler — slower compilation, heavy optimization).

JVM FLAGS — JIT Diagnostics

```
# Print when methods get JIT-compiled
-XX:+PrintCompilation

# Print inlining decisions
-XX:+UnlockDiagnosticVMOptions -XX:+PrintInlining

# Force C2 compilation threshold
-XX:CompileThreshold=10000

# Output native assembly (requires hsdis library)
-XX:+PrintAssembly
```

JIT Optimizations Explained

METHOD INLINING

The JIT replaces a method call with the actual body of the method. This eliminates the overhead of creating a new stack frame. It also unlocks further optimizations like constant folding on the inlined code.

JAVA — Inlining Example

```java
// Before JIT inlining:
int result = add(5, 3);

private int add(int a, int b) { return a + b; }

// After JIT inlining — the method call is GONE:
int result = 5 + 3;  // Further optimized to: int result = 8;
```

ESCAPE ANALYSIS

If the JIT can prove that an object never "escapes" the current method (is never returned or stored externally), it can allocate that object on the stack instead of the heap, eliminating GC pressure entirely. It can also perform **scalar replacement** — breaking the object into its constituent fields stored as local variables.

DEAD CODE ELIMINATION

If a branch is never taken after profiling, the JIT removes it. If a computed value is never used, the computation is removed.

LOOP UNROLLING

Loops with known iteration counts can be unrolled — the loop body is repeated multiple times with the loop overhead reduced. This exposes more parallelism for CPU pipelines.

> 💡 **PRODUCTION TIP: TIERED COMPILATION**
>
> Java 8+ uses Tiered Compilation by default (`-XX:+TieredCompilation`). Methods are first compiled by C1 (fast, minimal optimization) then promoted to C2 (slower, maximum optimization) if they remain hot. This gives fast startup AND fast steady-state throughput.

1.5 Stack Frames

Every method invocation creates a new **Stack Frame** on the thread's Java Stack. The frame contains three things: a **Local Variable Array** (method parameters + local vars), an **Operand Stack** (the working area where computations happen), and a **Frame Data** block (reference to constant pool, return address).

JAVA — Stack Frame Demo

```java
public int multiply(int a, int b) {
    int result = a * b;   // locals[0]=this, locals[1]=a, locals[2]=b, locals[3]=result
    return result;
}

// Bytecode sequence:
// iload_1       ← push a from local[1] onto operand stack
// iload_2       ← push b from local[2] onto operand stack
// imul          ← pop a and b, multiply, push result
// istore_3      ← pop result, store in local[3]
// iload_3       ← push result for return
// ireturn       ← return int from top of operand stack
```

CHAPTER 02

Memory Management & Garbage Collection

Heap generational structure, GC algorithms, G1GC deep dive, ZGC for low-latency, and production JVM tuning flags.

2.1 Heap Structure — Generational Hypothesis

The JVM heap is divided into generations based on the **Generational Hypothesis**: most objects die young. By separating short-lived objects (Young Gen) from long-lived ones (Old Gen), the GC can collect the Young Gen very frequently with low cost, and collect the Old Gen rarely.

Eden Space

New allocations land here

Survivor S0 / S1

Objects that survived minor GC

Old Generation

Long-lived objects

Object lifecycle: Eden → (Minor GC → age++) → Survivor → (age > 15) → Old Gen → (Major/Full GC)

Object Age and Tenuring

Each object carries a 4-bit **age counter** in its object header. When a Minor GC occurs, live objects in Eden are moved to a Survivor space and their age is incremented. When an object's age exceeds the tenuring threshold (default: 15), it is promoted to Old Gen. You can inspect and change this:

JVM FLAGS Heap Sizing & Tenuring

```
# Set initial and maximum heap size
-Xms512m -Xmx4g

# Set Young Generation size explicitly
-Xmn1g

# Ratio of Young to total heap (default: 2 means 1/3 of heap is Young)
-XX:NewRatio=2

# Object age at which it gets promoted to Old Gen
-XX:MaxTenuringThreshold=15

# Print object age distribution after each Minor GC
-XX:+PrintTenuringDistribution
```

2.2 Garbage Collection Algorithms

Serial GC

Single-threaded stop-the-world collector. Freezes all application threads, collects, resumes. Suitable only for small heaps or single-CPU environments. Never use in production server apps.

Parallel GC (Throughput Collector)

Multi-threaded Minor and Major GC. Maximizes throughput at the cost of latency. All application threads still stop during collection. Good for batch processing workloads where pause time is not critical.

CMS (Concurrent Mark Sweep) — Deprecated

Performs most of the Old Gen collection concurrently with application threads. Two short stop-the-world pauses (initial mark, remark) with concurrent marking and sweeping in between. Suffers from heap fragmentation (no compaction) leading to "concurrent mode failure". Removed in Java 14.

2.3 G1GC — Garbage First (Production Default)

G1 is the default GC since Java 9. It divides the heap into equal-sized **regions** (1–32MB each), rather than fixed-size generations. Each region can dynamically act as Eden, Survivor, Old, or Humongous (for large objects > 50% of region size). G1 predicts pause times and selects the regions with the most garbage ("Garbage First") to meet your pause time goal.

1. **MINOR GC (YOUNG COLLECTION)**
   Young regions are evacuated. Live objects copied to new Survivor/Old regions. Old region remembered sets track cross-region references.

2 **CONCURRENT MARKING CYCLE**

When heap occupancy exceeds IHOP (default 45%), G1 starts concurrent marking to identify live objects in Old regions.

3 **MIXED GC**

Both Young and selected Old regions are collected. G1 picks Old regions with highest garbage ratio (hence "Garbage First") to meet pause target.

4 **FULL GC (LAST RESORT)**

Serial single-threaded full compaction. G1's nemesis — avoid this! Caused by allocation rate exceeding collection rate.

JVM FLAGS G1GC Production Configuration

```
# Enable G1GC (default in Java 9+, but explicit for clarity)
-XX:+UseG1GC

# Target max GC pause time in ms (G1 will try to meet this)
-XX:MaxGCPauseMillis=200

# % of heap occupied before concurrent mark cycle starts
-XX:InitiatingHeapOccupancyPercent=45

# Number of concurrent GC threads
-XX:ConcGCThreads=4

# Number of parallel GC threads during stop-the-world
-XX:ParallelGCThreads=8

# Heap region size (must be power of 2, 1–32MB)
-XX:G1HeapRegionSize=16m

# Reserve % for emergency (prevents Full GC)
-XX:G1ReservePercent=10

# Enable string deduplication (reduces heap for String-heavy apps)
-XX:+UseStringDeduplication

# GC logging (Java 9+ unified logging)
-Xlog:gc*:file=/logs/gc.log:time,uptime,level,tags:filecount=10,filesize=10m
```

2.4 ZGC — Ultra-Low Latency (Java 15+ Production-Ready)

ZGC targets sub-millisecond pause times regardless of heap size (tested up to 16TB). It achieves this through **colored pointers** (metadata stored in object reference bits) and **load barriers** (code injected at

every object read to handle concurrent relocation). Nearly all GC work happens concurrently with application threads.

JVM FLAGS ZGC Configuration

```
# Enable ZGC
-XX:+UseZGC

# Return unused memory to OS after this many seconds
-XX:ZUncommitDelay=300

# Concurrent GC threads (set to ~25% of available CPUs)
-XX:ConcGCThreads=4

# Still want a pause time goal (advisory for ZGC)
-XX:MaxGCPauseMillis=10

# ZGC on Java 21+ — use generational ZGC for even better throughput
-XX:+UseZGC -XX:+ZGenerational
```

GC	Max Pause	Throughput	Best For
Serial	High	Low	CLI tools, single CPU
Parallel	Medium	High	Batch processing
G1GC	100–200ms	High	Most production apps
ZGC	<1ms	Good	Latency-sensitive APIs
Shenandoah	<10ms	Good	Low-latency alternative

2.5 Memory Leak Patterns to Watch For

🔴 Static Collections

Static `List` or `Map` that accumulates entries over time without ever removing them. The Class is never GC'd so neither is the collection.

🔴 ThreadLocal Leaks

`ThreadLocal` values in thread pools. If you set a value without calling `remove()`, the value lives as long as the thread (forever in a pool).

🔴 Unclosed Resources

JDBC connections, file streams, HTTP connections — if not closed, the backing native resources hold heap until GC + finalization.

🔴 Listener Registration

Registering event listeners or callbacks and never deregistering. The listener holds a reference preventing the subject from being GC'd.

⚠️ **OUTOFMEMORYERROR TYPES**

java.lang.OutOfMemoryError: Java heap space — heap exhausted. Increase -Xmx or fix leak. **java.lang.OutOfMemoryError: Metaspace** — class metadata space exhausted; tune -XX:MaxMetaspaceSize. **java.lang.OutOfMemoryError: GC overhead limit exceeded** — JVM spending >98% time in GC. Critical leak or undersized heap.

CHAPTER 03

Spring IoC & Dependency Injection

Container internals, BeanFactory vs ApplicationContext, the full bean lifecycle, DI patterns, scopes, and circular dependency resolution.

3.1 Inversion of Control — The Core Concept

In traditional programming, your code creates its own dependencies: `UserService service = new UserService(new UserRepository())`. With IoC, you invert this — you declare what you need, and the **container** creates and injects it. Your code never calls `new` for Spring-managed beans.

This decouples your classes from their dependencies, making them independently testable, replaceable, and configurable without recompilation.

3.2 BeanFactory vs ApplicationContext

`BeanFactory` is the root interface of the Spring container — it provides basic DI and lazy bean instantiation. `ApplicationContext` extends it and adds enterprise features: event publishing, internationalization, eager bean initialization, AOP integration, and environment abstraction. In practice, always use `ApplicationContext` (or its sub-interfaces like `AnnotationConfigApplicationContext`).

Feature	BeanFactory	ApplicationContext
Bean instantiation	Lazy (on first request)	Eager (at startup) + Lazy opt-in
Event publishing	❌	✅ ApplicationEvent
i18n MessageSource	❌	✅
AOP auto-proxying	❌	✅
Environment/Profiles	❌	✅
@PostConstruct/@PreDestroy	❌	✅

3.3 BeanDefinition — What Spring Really Stores

Before a bean exists, Spring stores a `BeanDefinition` — a recipe for creating the bean. It contains: the bean class, scope (singleton/prototype), constructor arguments, property values, initialization method name, destroy method name, lazy-init flag, and dependency relationships.

The `BeanDefinitionRegistry` stores all definitions. During startup, `BeanDefinitionReader` implementations scan your code (XML, annotations, Java config) and populate the registry.

3.4 Full Bean Creation Lifecycle

1. **BEANDEFINITION LOADING**

   Spring scans `@Component`, `@Service`, `@Repository`, `@Controller`, and `@Bean` methods. Each becomes a `BeanDefinition` in the registry.

2. **BEANFACTORYPOSTPROCESSOR**

   Processors like `PropertySourcesPlaceholderConfigurer` run and can modify `BeanDefinition` objects before any beans are created (e.g., resolve `\${property.value}` placeholders).

3. **BEAN INSTANTIATION**

   Spring creates the bean instance using the constructor or factory method. If constructor injection is used, all constructor dependencies must already be available.

4. **DEPENDENCY POPULATION**

   Setter or field injection dependencies are injected. Spring resolves dependencies by type (then name if ambiguous).

5. **BEANPOSTPROCESSOR — BEFORE INIT**

   `postProcessBeforeInitialization()` runs. This is where `@Autowired` field injection actually happens (via `AutowiredAnnotationBeanPostProcessor`).

6. **INITIALIZATION CALLBACKS**

   `@PostConstruct` method runs, then `afterPropertiesSet()` if the bean implements `InitializingBean`, then the custom `init-method`.

7. **BEANPOSTPROCESSOR — AFTER INIT**

   `postProcessAfterInitialization()` runs. AOP proxy creation happens here — the returned object may be a proxy wrapping the original bean.

8. **BEAN ADDED TO SINGLETON CACHE**

   The fully initialized bean is placed in the singleton cache (a `ConcurrentHashMap`). Future calls to `getBean()` return this cached instance.

9. **DESTRUCTION (ON CONTEXT CLOSE)**

   `@PreDestroy` runs, then `destroy()` if implementing `DisposableBean`, then custom `destroy-method`.

3.5 Dependency Injection Types

✅ CONSTRUCTOR INJECTION (RECOMMENDED)

Dependencies injected via constructor. Makes all required dependencies explicit and mandatory. The bean is *immutable* after creation — you can mark fields `final`. Works perfectly with `@RequiredArgsConstructor` from Lombok. Easy to test — just call the constructor directly with mocks.

❌ FIELD INJECTION (AVOID)

Uses `@Autowired` directly on private fields. Hidden dependencies — you cannot see what a class needs without reading every field. Cannot be used with `final`. Requires reflection — breaks encapsulation. Testing requires either Spring context or reflection hacks.

JAVA — Constructor Injection — Best Practice

```java
@Service
public class OrderService {

    private final OrderRepository orderRepo;
    private final PaymentService paymentService;
    private final NotificationService notificationService;

    // Spring auto-detects single constructor — @Autowired optional
    public OrderService(OrderRepository orderRepo,
                        PaymentService paymentService,
                        NotificationService notificationService) {
        this.orderRepo = orderRepo;
        this.paymentService = paymentService;
        this.notificationService = notificationService;
    }
}

// Lombok equivalent — generates the constructor automatically:
@Service
@RequiredArgsConstructor
public class OrderService {
    private final OrderRepository orderRepo;
    private final PaymentService paymentService;
    private final NotificationService notificationService;
    // Constructor generated by Lombok!
}
```

3.6 Bean Scopes

Scope	Description	Created	Destroyed
singleton	One instance per Spring container (default)	Container startup	Container shutdown
prototype	New instance every time the bean is requested	Each request	Caller's responsibility
request	One per HTTP request (web only)	HTTP request start	HTTP request end
session	One per HTTP session (web only)	Session creation	Session timeout
application	One per ServletContext (web only)	App startup	App shutdown

3.7 Circular Dependency

Circular dependency occurs when A needs B and B needs A. With constructor injection, Spring **cannot** resolve this (correctly) — it throws `BeanCurrentlyInCreationException`. The fix: redesign your architecture so A and B don't mutually depend on each other (introduce a third service, or use events). For setter injection, Spring can resolve it using a three-level cache, but this is a code smell — avoid it.

JAVA

@PostConstruct & @PreDestroy

```java
@Component
public class CacheWarmupService {

    private final UserRepository userRepo;
    private final CacheManager cacheManager;

    public CacheWarmupService(UserRepository userRepo, CacheManager cacheManager) {
        this.userRepo = userRepo;
        this.cacheManager = cacheManager;
    }

    @PostConstruct  // Runs AFTER all dependencies injected, BEFORE bean is used
    public void warmUpCache() {
        List<User> users = userRepo.findAll();
        cacheManager.getCache("users").put("all", users);
        System.out.println("Cache warmed up with " + users.size() + " users");
    }

    @PreDestroy  // Runs BEFORE bean destroyed, use for cleanup
    public void evictCache() {
        cacheManager.getCache("users").clear();
        System.out.println("Cache evicted cleanly");
    }
}
```

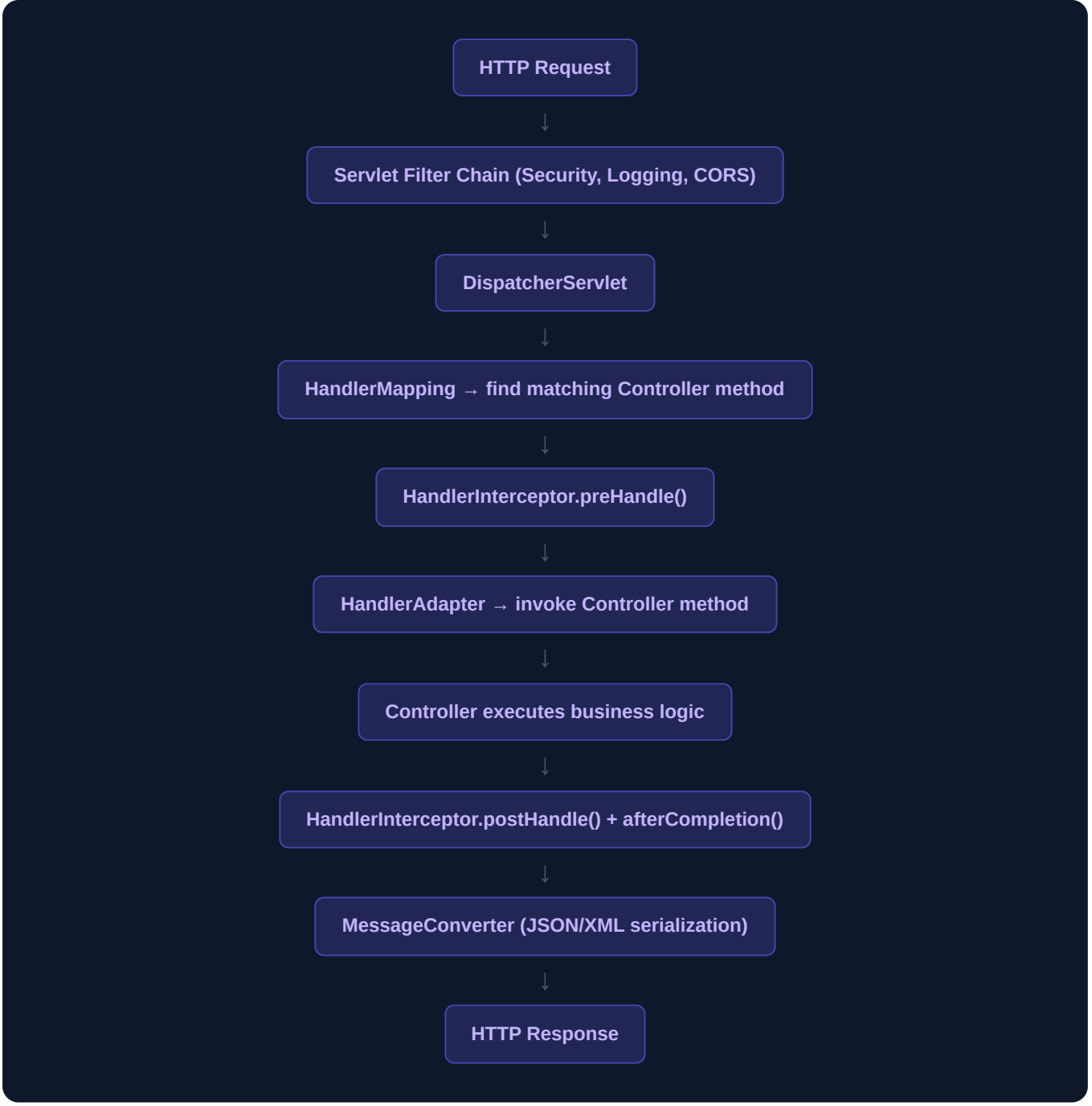
CHAPTER 04

Spring MVC — Internal Request Pipeline

DispatcherServlet as the front controller, HandlerMapping, HandlerAdapter, Filters, Interceptors, exception handling with @ControllerAdvice, and response rendering.

4.1 DispatcherServlet — Front Controller Pattern

Spring MVC implements the **Front Controller Pattern**. Every HTTP request to your application is handled by a single servlet — the `DispatcherServlet`. It acts as the central command-and-control hub, delegating to specialized components.



4.2 HandlerMapping

`HandlerMapping` is responsible for mapping an incoming HTTP request (URL + method + headers) to the correct handler (controller method). Spring Boot registers `RequestMappingHandlerMapping` which scans all `@RequestMapping` (and `@GetMapping`, `@PostMapping`, etc.) annotations and builds a routing table at startup.

4.3 HandlerAdapter

Once the handler is found, the `HandlerAdapter` knows how to invoke it. `RequestMappingHandlerAdapter` handles the complex task of resolving method parameters — it iterates through all registered `HandlerMethodArgumentResolver` instances to determine how to provide values for each parameter (from path, query string, request body, headers, session, etc.).

JAVA RestController — Full Example

```java
@RestController
@RequestMapping("/api/v1/users")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @GetMapping("/{id}")
    public ResponseEntity<UserDto> getUser(@PathVariable Long id) {
        return userService.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<UserDto> createUser(
            @RequestBody @Valid CreateUserRequest request) {
        UserDto created = userService.create(request);
        URI location = ServletUriComponentsBuilder
            .fromCurrentRequest()
            .path("/{id}")
            .buildAndExpand(created.getId())
            .toUri();
        return ResponseEntity.created(location).body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<UserDto> updateUser(
            @PathVariable Long id,
            @RequestBody @Valid UpdateUserRequest request) {
        return ResponseEntity.ok(userService.update(id, request));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
        userService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

4.4 Filters vs Interceptors

✅ SERVLET FILTER (JAVAX.SERVLET.FILTER)

Operates at the Servlet container level — *before* DispatcherServlet. Can intercept any HTTP request, including static resources. Used for security, CORS, rate limiting, request logging, compression. Spring Security's filter chain is implemented here.

✅ HANDLERINTERCEPTOR (SPRING-LEVEL)

Operates *inside* DispatcherServlet, after handler mapping. Has access to the handler (controller method) and model. `preHandle()`, `postHandle()`, `afterCompletion()`. Used for auth checks, performance monitoring, locale detection.

JAVA — HandlerInterceptor — Request Timing

```java
@Component
public class TimingInterceptor implements HandlerInterceptor {

    private static final String START_TIME = "startTime";

    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res,
                             Object handler) {
        req.setAttribute(START_TIME, System.currentTimeMillis());
        return true;  // return false to abort request processing
    }

    @Override
    public void afterCompletion(HttpServletRequest req, HttpServletResponse res,
                                Object handler, Exception ex) {
        long duration = System.currentTimeMillis() - (Long) req.getAttribute(START_TIME);
        String uri = req.getRequestURI();
        log.info("[{}] {} → {}ms", res.getStatus(), uri, duration);
    }
}
```

4.5 Global Exception Handling with @ControllerAdvice

Instead of wrapping every controller method in try-catch, use `@ControllerAdvice` (or `@RestControllerAdvice`) to define centralized exception handlers. The `@ExceptionHandler` method is invoked whenever the specified exception propagates out of any controller.

JAVA Global Exception Handler

```java
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("NOT_FOUND", ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
            MethodArgumentNotValidException ex) {
        List<String> errors = ex.getBindingResult().getFieldErrors()
            .stream()
            .map(e -> e.getField() + ": " + e.getDefaultMessage())
            .collect(Collectors.toList());
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ErrorResponse("VALIDATION_FAILED", errors.toString()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAll(Exception ex) {
        log.error("Unhandled exception", ex);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("INTERNAL_ERROR", "An unexpected error occurred"));
    }
}
```

CHAPTER 05

Spring Boot Internals

Auto-configuration magic, spring.factories / AutoConfiguration.imports, conditional beans, Actuator, Profiles, and embedded server configuration.

5.1 SpringApplication.run() — What Actually Happens

When you call `SpringApplication.run(App.class, args)`, a precise sequence unfolds:

1. **DETERMINE APPLICATION TYPE**
   Is classpath a SERVLET app, REACTIVE (WebFlux), or NONE? This determines which `ApplicationContext` type to create.

2. **LOAD SPRINGAPPLICATIONRUNLISTENERS**
   Via SPI — these receive lifecycle events throughout startup.

3. **PREPARE ENVIRONMENT**
   Creates `ConfigurableEnvironment`, loads all property sources: system properties, env vars, `application.properties`, `application.yml`, etc. Activates profiles.

4. **CREATE APPLICATIONCONTEXT**
   Creates the appropriate context type. Registers the main class as a configuration source.

5. **REFRESH CONTEXT**
   Invokes `AbstractApplicationContext.refresh()` — the heart of Spring. Scans components, processes auto-configuration, creates all singleton beans.

6. **CALL APPLICATIONRUNNERS**
   Any beans implementing `ApplicationRunner` or `CommandLineRunner` are invoked. Good for post-startup initialization.

5.2 Auto-Configuration Deep Dive

`@SpringBootApplication` is a composed annotation of `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan`. The `@EnableAutoConfiguration` triggers the auto-configuration mechanism.

In Spring Boot 2.x, auto-configuration classes were listed in `META-INF/spring.factories`. In Spring Boot 3.x, they moved to `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (one class per line, simpler format).

TEXT — META-INF/spring/...AutoConfiguration.imports (Spring Boot 3)

```text
org.springframework.boot.autoconfigure.data.jpa.JpaRepositoriesAutoConfiguration
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
org.springframework.boot.autoconfigure.web.servlet.WebMvcAutoConfiguration
org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration
# ... hundreds more
```

Each auto-configuration class is a `@Configuration` class loaded by `AutoConfigurationImportSelector`. But they're all guarded by `@Conditional` annotations — they only take effect when specific conditions are met:

Annotation	Condition
`@ConditionalOnClass`	Class exists on classpath (e.g., DataSource.class)
`@ConditionalOnMissingBean`	No bean of this type defined by user
`@ConditionalOnProperty`	Property has specific value in config
`@ConditionalOnWebApplication`	Running as a web application
`@ConditionalOnBean`	Specific bean already exists in context
`@ConditionalOnMissingClass`	Class NOT on classpath

JAVA — Custom Auto-Configuration

```java
@AutoConfiguration
@ConditionalOnClass(DataSource.class)
@ConditionalOnProperty(name = "myapp.cache.enabled", havingValue = "true", matchIfMissing = fals
public class CacheAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public CacheManager cacheManager() {
        return new ConcurrentMapCacheManager();
    }
}
```

5.3 Configuration Hierarchy (Property Priority)

Spring Boot applies a specific property source priority (higher overrides lower):

1. CLI args (highest)
2. System properties
3. OS env vars
4. application-{profile}.yml
5. application.yml
6. @PropertySource
7. Default values (lowest)

5.4 Profiles

YAML — application-prod.yml

```yaml
spring:
  datasource:
    url: jdbc:postgresql://prod-db:5432/myapp
    username: \${DB_USER}       # from environment variable
    password: \${DB_PASSWORD}

  jpa:
    show-sql: false            # disable SQL logging in prod

logging:
  level:
    root: WARN
    com.myapp: INFO
```

5.5 Actuator — Production Monitoring

Spring Boot Actuator exposes production-ready endpoints for monitoring, health checking, and introspection. Always used with Prometheus, Grafana, and APM tools in production.

YAML — Actuator Configuration

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,env
  endpoint:
    health:
      show-details: when_authorized  # hide sensitive details by default
  metrics:
    export:
      prometheus:
        enabled: true
```

Endpoint	URL	Purpose
Health	/actuator/health	DB, disk, external service health checks
Metrics	/actuator/metrics	JVM, HTTP, DB metrics
Prometheus	/actuator/prometheus	Metrics in Prometheus format (scrape target)
Env	/actuator/env	Environment properties (sensitive — secure this!)
Beans	/actuator/beans	All registered Spring beans
Loggers	/actuator/loggers	View/change log levels at runtime

CHAPTER 06

HikariCP — Production Connection Pooling

Why connection pooling matters, Hikari's architecture advantages, optimal pool sizing formula, monitoring, and production configuration patterns.

6.1 Why Connection Pooling?

Establishing a JDBC database connection is expensive — it involves a network TCP handshake, database authentication, session initialization, and memory allocation on both sides. Creating a new connection for every database request would be catastrophically slow. Connection pooling maintains a set of pre-established connections that are borrowed and returned by application threads.

6.2 Why HikariCP?

HikariCP is the default connection pool in Spring Boot. It is consistently the fastest Java connection pool in benchmarks. Key architectural decisions that make it fast: it uses a lock-free `ConcurrentBag` data structure for the pool, eliminates proxy wrappers around Statements (direct delegation instead), uses aggressive bytecode optimization, and avoids thread synchronization wherever possible.

6.3 Pool Sizing — The Formula

Pool size is one of the most misunderstood configurations. Bigger is NOT better. The optimal formula, derived from database and OS theory:

📐 THE FORMULA (HIKARICP TEAM RECOMMENDATION)

pool_size = (core_count * 2) + effective_spindle_count

For a modern SSD/NVMe disk server, effective_spindle_count = 1. For a 4-core CPU app server: pool_size = (4 × 2) + 1 = **9**. Too large a pool wastes memory and causes context switching. Too small causes connection wait timeouts.

PROPERTIES application.properties — HikariCP Production Config

```properties
# Maximum number of connections in the pool
spring.datasource.hikari.maximum-pool-size=20

# Minimum idle connections maintained (set equal to max for fixed pool)
spring.datasource.hikari.minimum-idle=5

# How long a connection can sit idle before being removed (ms)
spring.datasource.hikari.idle-timeout=300000

# Max time to wait for a connection from pool before throwing (ms)
spring.datasource.hikari.connection-timeout=30000

# Max lifetime of a connection in pool — MUST be less than db wait_timeout (ms)
spring.datasource.hikari.max-lifetime=1800000

# Query to validate connections on borrow
spring.datasource.hikari.connection-test-query=SELECT 1

# Pool name (shows in JMX and logs)
spring.datasource.hikari.pool-name=MyAppHikariPool

# Enable JMX monitoring
spring.datasource.hikari.register-mbeans=true

# Enable Prometheus metrics via Micrometer
spring.datasource.hikari.metrics-registry=hikaricp
```

6.4 Monitoring the Pool

HikariCP integrates with Micrometer to expose pool metrics automatically when Actuator is on the classpath. Key metrics to track in Grafana:

Metric	Meaning	Alert If
hikaricp.connections.active	Connections currently in use	Consistently = max-pool-size
hikaricp.connections.pending	Threads waiting for connection	> 0 for sustained period
hikaricp.connections.timeout.total	Times a connection wait timed out	> 0
hikaricp.connections.acquire	Time to get connection from pool (p99)	> 100ms p99
hikaricp.connections.creation	Time to create a new connection	High = DB overloaded

⚠️ **MAX-LIFETIME MUST BE LESS THAN DB TIMEOUT**

MySQL default `wait_timeout` is 8 hours (28,800 seconds). PostgreSQL default is unlimited. If your pool holds a connection longer than the database's timeout, the DB silently closes it. HikariCP will try to use a dead connection, fail, then discard it. Set `max-lifetime` to at least 30 seconds less than the database's timeout.

6.5 Detecting Connection Leaks

PROPERTIES Connection Leak Detection

```
# Log a warning if a connection is held longer than this (ms)
# 0 = disabled. Set to something like 10000ms in development
spring.datasource.hikari.leak-detection-threshold=10000
```

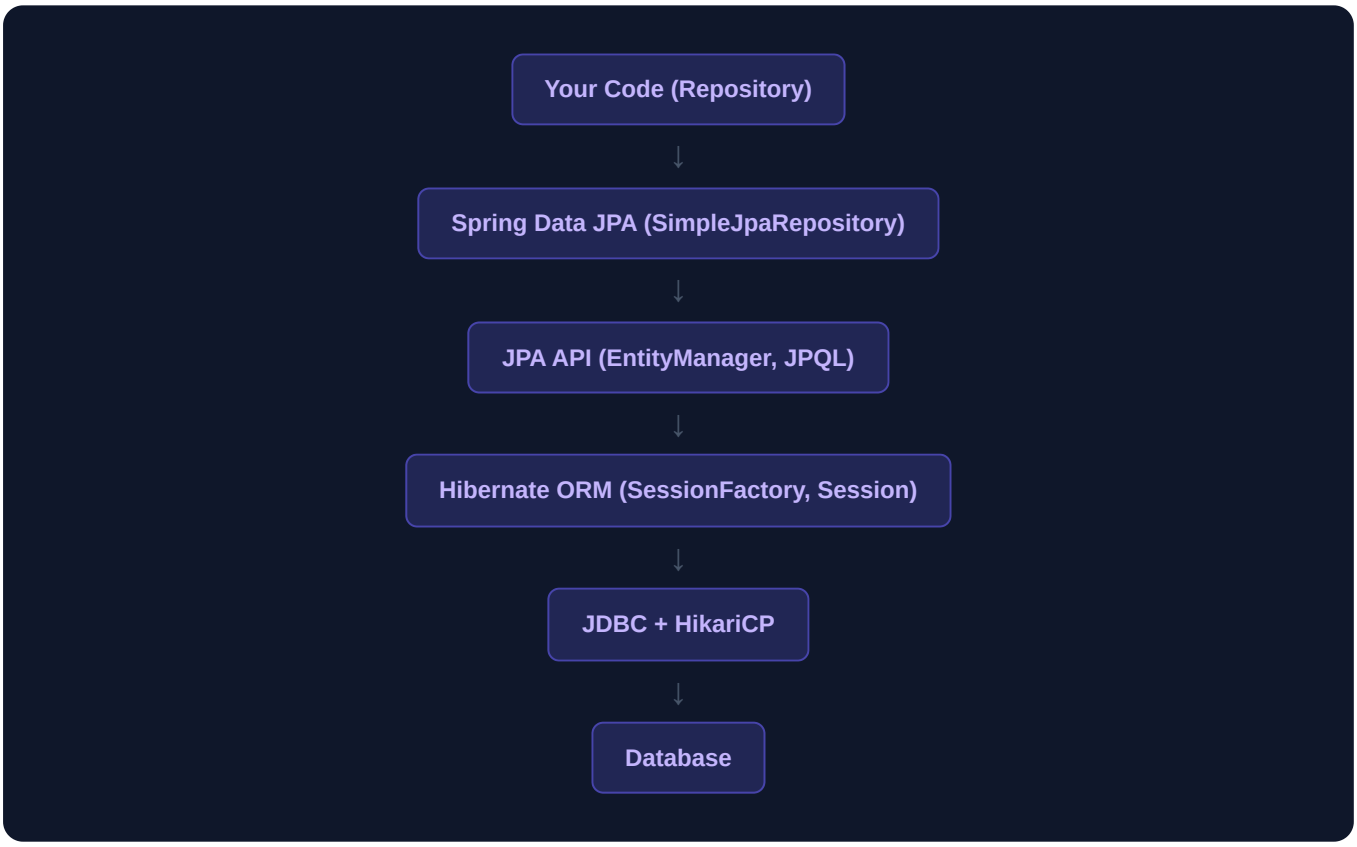
CHAPTER 07

JPA & Hibernate Internals

Session, Persistence Context, First/Second Level Cache, dirty checking, lazy loading, N+1 problem, entity states, and JPQL/Criteria queries.

7.1 Architecture Overview

JPA (Jakarta Persistence API) is a specification — a set of interfaces and annotations. Hibernate is the most widely used implementation. Spring Data JPA sits on top, providing repository abstractions. Understanding the layering is critical.



7.2 Entity States

State	Description	Tracked by PC?
Transient	Object created with `new`, not associated with any persistence context	No
Managed (Persistent)	Returned from `find()`/`persist()`/`merge()`. Changes are automatically synchronized	Yes
Detached	Was managed, but Session/EntityManager closed. Changes NOT tracked	No
Removed	Marked for deletion via `remove()`. Will be deleted on flush	Yes (until flush)

7.3 Dirty Checking — How Hibernate Auto-Tracks Changes

When you load an entity within a transaction, Hibernate snapshots its state. At transaction commit (or explicit flush), Hibernate compares the current state against the snapshot. Any differences trigger automatic `UPDATE` SQL. You never call `save()` on a managed entity — just modify it and let Hibernate handle the rest.

JAVA — Dirty Checking Demo

```java
@Transactional
public void updateUserEmail(Long id, String newEmail) {
    User user = userRepo.findById(id).orElseThrow();
    // User is now MANAGED — in the Persistence Context

    user.setEmail(newEmail);
    // NO save() needed! Hibernate detects the email changed.
    // On @Transactional commit → Hibernate flushes → UPDATE user SET email=? WHERE id=?
}
```

7.4 First-Level Cache (Persistence Context Cache)

Every `EntityManager` (Session) has a first-level cache — a simple identity map. Within the same transaction, loading the same entity twice by ID returns the same object instance — no second database query. This cache is scoped to the `EntityManager` lifetime (typically one request/transaction).

7.5 Second-Level Cache

The second-level (L2) cache is shared across sessions and optionally persisted to a distributed cache like Redis or Hazelcast. It reduces database hits for frequently-read, rarely-changed data (reference data, user preferences, product catalogs).

JAVA — Second-Level Cache Configuration

```java
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Country {
    @Id
    private String code;
    private String name;
}
```

PROPERTIES — L2 Cache Config with Ehcache

```properties
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
spring.jpa.properties.hibernate.cache.region.factory_class=org.hibernate.cache.jcache.JCacheRegi
spring.jpa.properties.javax.cache.provider=org.ehcache.jsr107.EhcacheCachingProvider
```

7.6 The N+1 Problem — The Most Common JPA Killer

N+1 occurs when fetching a list of N parent entities, then separately querying for each one's lazily-loaded collection — resulting in 1 query for the list + N queries for the children. With 1000 users, that's 1001 queries for what should be 1.

JAVA N+1 Problem & Fixes

```java
// ❌ N+1 PROBLEM: Triggers 1 + N queries
List<User> users = userRepo.findAll();
users.forEach(u -> u.getPosts().size());  // Lazy load posts for EACH user!

// ✅ FIX 1: JOIN FETCH in JPQL
@Query("SELECT u FROM User u LEFT JOIN FETCH u.posts")
List<User> findAllWithPosts();

// ✅ FIX 2: @EntityGraph (cleaner — no JPQL needed)
@EntityGraph(attributePaths = {"posts", "roles"})
@Query("SELECT u FROM User u")
List<User> findAllWithPostsAndRoles();

// ✅ FIX 3: Batch fetching (lazy load but in batches)
@OneToMany(fetch = FetchType.LAZY)
@BatchSize(size = 25)
private List<Post> posts;

// In properties (global batch size):
// spring.jpa.properties.hibernate.default_batch_fetch_size=25
```

7.7 Relationships — Mapping Best Practices

JAVA Entity Relationship Mapping

```java
@Entity
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    // Many posts belong to one user — FK on post table
    @ManyToOne(fetch = FetchType.LAZY)  // ALWAYS lazy for @ManyToOne
    @JoinColumn(name = "user_id")
    private User author;

    // One post has many comments
    @OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Comment> comments = new ArrayList<>();

    // Helper method for bidirectional consistency
    public void addComment(Comment comment) {
        comments.add(comment);
        comment.setPost(this);
    }
}
```

7.8 Projections — Fetch Only What You Need

JAVA

Interface & Class Projections

```java
// Interface-based projection (Spring Data generates implementation)
public interface UserSummary {
    Long getId();
    String getName();
    String getEmail();
}

@Query("SELECT u.id as id, u.name as name, u.email as email FROM User u")
List<UserSummary> findUserSummaries();

// DTO projection with constructor expression (fully type-safe)
@Query("SELECT new com.app.dto.UserDto(u.id, u.name, u.email) FROM User u WHERE u.active = true"
List<UserDto> findActiveUserDtos();
```

CHAPTER 08

Spring Security + JWT

SecurityFilterChain, authentication flow, JWT token generation and validation, role-based access control, method security, and CORS configuration.

8.1 Spring Security Filter Chain

Spring Security is implemented as a chain of Servlet Filters. Every HTTP request passes through this chain before reaching your controllers. Each filter has a specific responsibility. You can add, remove, or replace filters.

Filter	Responsibility
SecurityContextPersistenceFilter	Loads SecurityContext from storage (session/stateless)
UsernamePasswordAuthenticationFilter	Handles /login form submissions
JwtAuthenticationFilter (custom)	Validates JWT and sets authentication in context
ExceptionTranslationFilter	Converts AuthenticationException → 401, AccessDeniedException → 403
FilterSecurityInterceptor	Final access control check against rules

8.2 SecurityFilterChain Configuration (Spring Boot 3.x)

JAVA SecurityConfig.java

```java
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;
    private final UserDetailsService userDetailsService;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())  // JWT is stateless, no CSRF needed
            .sessionManagement(sm -> sm
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED))
                .accessDeniedHandler((req, res, e) ->
                    res.sendError(HttpServletResponse.SC_FORBIDDEN, "Access Denied")))
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);  // cost factor 12
    }

    @Bean
    public AuthenticationManager authManager(AuthenticationConfiguration config)
            throws Exception {
        return config.getAuthenticationManager();
    }
}
```

8.3 JWT — Full Implementation

JAVA JwtService.java (using jjwt 0.12+)

```java
@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.expiration:86400000}")  // 24h default
    private long jwtExpiration;

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secretKey));
    }

    public String generateToken(UserDetails userDetails) {
        return generateToken(Map.of(), userDetails);
    }

    public String generateToken(Map<String, Object> extraClaims, UserDetails userDetails) {
        return Jwts.builder()
            .claims(extraClaims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + jwtExpiration))
            .signWith(getSigningKey())
            .compact();
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
    }

    private boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
```

```
            .parseSignedClaims(token)
            .getPayload();
        return claimsResolver.apply(claims);
    }
}
```

JAVA JwtAuthFilter.java

```java
@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        final String jwt = authHeader.substring(7);
        final String userEmail = jwtService.extractUsername(jwt);

        if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null)
            UserDetails userDetails = userDetailsService.loadUserByUsername(userEmail);

            if (jwtService.isTokenValid(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(new WebAuthenticationDetailsSource()
                    .buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

8.4 Method-Level Security

JAVA

Method Security Annotations

```java
@Service
public class UserService {

    @PreAuthorize("hasRole('ADMIN')")
    public List<User> getAllUsers() { ... }

    @PreAuthorize("#id == authentication.principal.id or hasRole('ADMIN')")
    public User getUserById(Long id) { ... }

    @PostAuthorize("returnObject.owner == authentication.name")
    public Document getDocument(Long id) { ... }

    @Secured({"ROLE_ADMIN", "ROLE_MANAGER"})
    public void deleteUser(Long id) { ... }
}
```

CHAPTER 09

Redis — Caching & Rate Limiting

Spring Cache abstraction, Cache-Aside pattern, TTL strategies, distributed rate limiting with sliding window, and Redis configuration.

9.1 Why Redis?

Redis is an in-memory data structure store operating at sub-millisecond latency. For a backend service, Redis is the standard solution for: response caching, session storage, distributed rate limiting, pub/sub messaging, distributed locks, and leaderboards. Spring Boot has first-class Redis support via Spring Data Redis and the Spring Cache abstraction.

9.2 Spring Cache Abstraction

JAVA

Redis Cache Configuration

```java
@Configuration
public class CacheConfig {

    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory factory) {
        RedisCacheConfiguration defaultConfig = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(30))
            .serializeKeysWith(RedisSerializationContext.SerializationPair
                .fromSerializer(new StringRedisSerializer()))
            .serializeValuesWith(RedisSerializationContext.SerializationPair
                .fromSerializer(new GenericJackson2JsonRedisSerializer()))
            .disableCachingNullValues();

        Map<String, RedisCacheConfiguration> cacheConfigs = new HashMap<>();
        cacheConfigs.put("users", defaultConfig.entryTtl(Duration.ofHours(1)));
        cacheConfigs.put("products", defaultConfig.entryTtl(Duration.ofMinutes(10)));

        return RedisCacheManager.builder(factory)
            .cacheDefaults(defaultConfig)
            .withInitialCacheConfigurations(cacheConfigs)
            .build();
    }
}
```

JAVA Cache Annotations Usage

```java
@Service
public class ProductService {

    // Cache result. Key: "products::123"
    @Cacheable(value = "products", key = "#id")
    public ProductDto getProduct(Long id) {
        return productRepo.findById(id).map(this::toDto).orElseThrow();
    }

    // Update cache entry on save
    @CachePut(value = "products", key = "#result.id")
    public ProductDto updateProduct(Long id, UpdateProductRequest req) {
        Product product = productRepo.findById(id).orElseThrow();
        product.setName(req.getName());
        return toDto(productRepo.save(product));
    }

    // Evict cache entry on delete
    @CacheEvict(value = "products", key = "#id")
    public void deleteProduct(Long id) {
        productRepo.deleteById(id);
    }

    // Evict ALL product cache entries
    @CacheEvict(value = "products", allEntries = true)
    public void clearProductCache() {}
}
```

9.3 Distributed Rate Limiting with Redis

Rate limiting protects your API from abuse and ensures fair resource allocation. Using Redis for rate limiting makes it work across multiple instances of your service. The **sliding window** algorithm uses a sorted set: members are request timestamps, scores are timestamps, and we count entries within the last window period.

JAVA Redis Rate Limiter — Sliding Window

```java
@Component
@RequiredArgsConstructor
public class RateLimiter {

    private final StringRedisTemplate redisTemplate;

    private static final int MAX_REQUESTS = 100;
    private static final long WINDOW_MS = 60_000;  // 1 minute

    public boolean isAllowed(String clientId) {
        long now = System.currentTimeMillis();
        long windowStart = now - WINDOW_MS;
        String key = "rate_limit:" + clientId;

        return (Boolean) redisTemplate.execute((RedisCallback<Boolean>) conn -> {
            ZSetOperations<String, String> zSet = redisTemplate.opsForZSet();

            // Remove timestamps outside the window
            zSet.removeRangeByScore(key, 0, windowStart);

            // Count remaining requests in window
            Long count = zSet.zCard(key);

            if (count != null && count < MAX_REQUESTS) {
                // Add this request's timestamp
                zSet.add(key, String.valueOf(now), now);
                redisTemplate.expire(key, Duration.ofMillis(WINDOW_MS));
                return true;
            }
            return false;
        });
    }
}

// In a filter:
if (!rateLimiter.isAllowed(clientIp)) {
    response.setStatus(429);  // Too Many Requests
    response.getWriter().write("Rate limit exceeded. Try again in 60 seconds.");
    return;
}
```

9.4 Cache-Aside Pattern (Manual Redis)

JAVA

Manual Cache-Aside

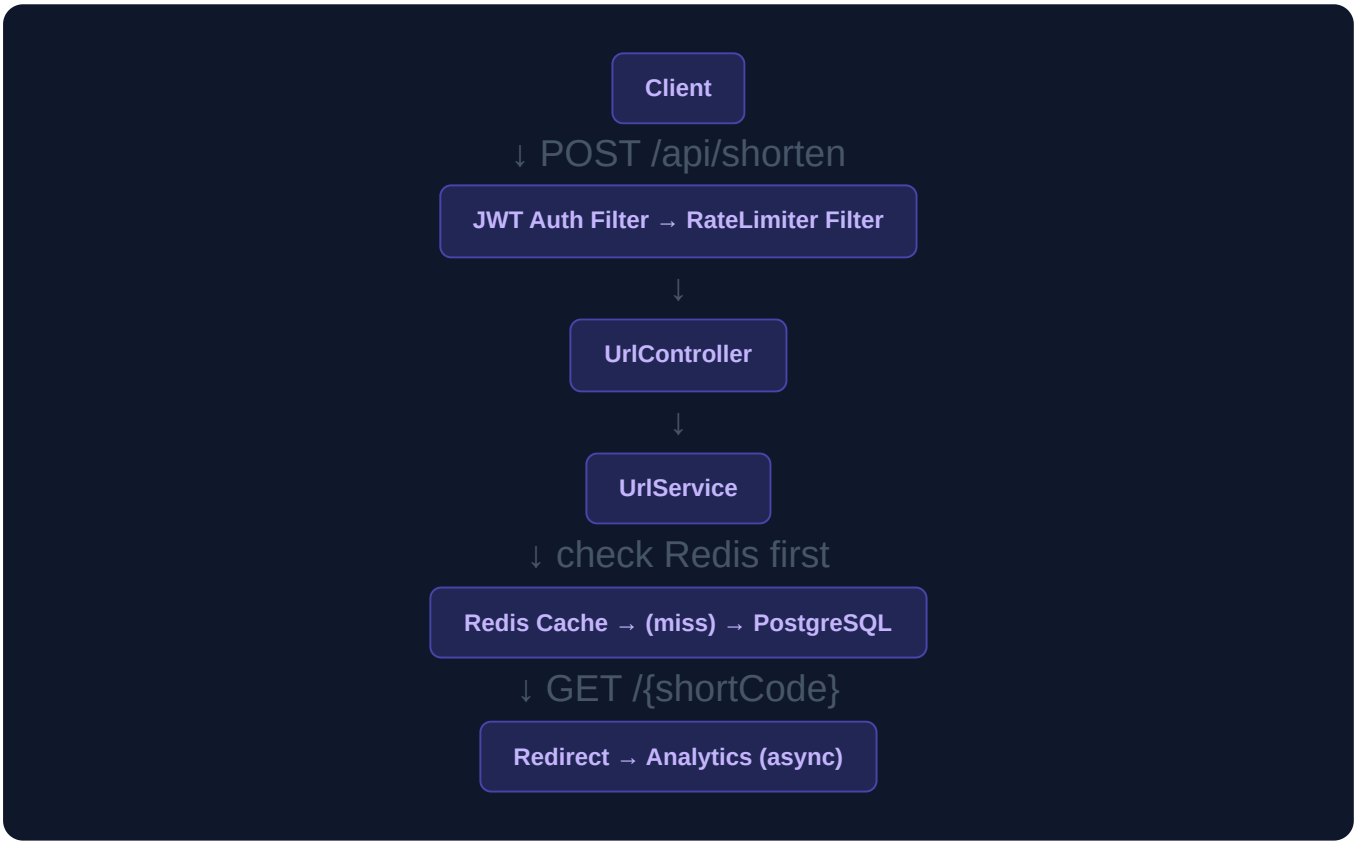
```java
@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepo;
    private final RedisTemplate<String, UserDto> redisTemplate;
    private static final Duration TTL = Duration.ofHours(1);

    public UserDto getUserById(Long id) {
        String key = "user:" + id;

        // 1. Try cache first
        UserDto cached = redisTemplate.opsForValue().get(key);
        if (cached != null) {
            return cached;
        }

        // 2. Cache miss — load from DB
        UserDto user = userRepo.findById(id)
            .map(this::toDto)
            .orElseThrow(() -> new ResourceNotFoundException("User not found: " + id));

        // 3. Store in cache for next request
        redisTemplate.opsForValue().set(key, user, TTL);

        return user;
    }

    public void updateUser(Long id, UpdateUserRequest req) {
        // Update DB first (write-through or invalidate cache)
        User user = userRepo.findById(id).orElseThrow();
        user.setName(req.getName());
        userRepo.save(user);

        // Invalidate cache — next read will refresh from DB
        redisTemplate.delete("user:" + id);
    }
}
```

CHAPTER 10

Project: Production URL Shortener

End-to-end implementation with full security, Redis caching, rate limiting, analytics tracking, and deployment-ready architecture.

10.1 System Architecture



10.2 Domain Entity

JAVA Url.java

```java
@Entity
@Table(name = "urls", indexes = {
    @Index(name = "idx_short_code", columnList = "shortCode", unique = true),
    @Index(name = "idx_user_id", columnList = "userId")
})
@EntityListeners(AuditingEntityListener.class)
public class Url {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 2048)
    private String originalUrl;

    @Column(nullable = false, unique = true, length = 10)
    private String shortCode;

    @Column(nullable = false)
    private Long userId;

    @CreatedDate
    private LocalDateTime createdAt;

    @Column
    private LocalDateTime expiresAt;

    @Column(columnDefinition = "bigint default 0")
    private Long clickCount = 0L;

    public boolean isExpired() {
        return expiresAt != null && LocalDateTime.now().isAfter(expiresAt);
    }
}
```

10.3 Service Layer

JAVA UrlService.java

```java
@Service
@RequiredArgsConstructor
@Slf4j
public class UrlService {

    private final UrlRepository urlRepo;
    private final RedisTemplate<String, String> redisTemplate;
    private final AnalyticsService analyticsService;
    private static final int CODE_LENGTH = 7;

    @Transactional
    public UrlResponse shorten(ShortenRequest request, Long userId) {
        String code = generateUniqueCode();

        Url url = new Url();
        url.setOriginalUrl(request.getUrl());
        url.setShortCode(code);
        url.setUserId(userId);
        url.setExpiresAt(request.getExpiresAt());

        Url saved = urlRepo.save(url);

        // Cache the mapping for fast redirect lookups
        redisTemplate.opsForValue().set(
            "url:" + code, request.getUrl(), Duration.ofHours(24));

        log.info("Created short URL: {} → {}", code, request.getUrl());
        return UrlResponse.from(saved);
    }

    public String resolve(String code, String clientIp, String userAgent) {
        // 1. Try Redis first (O(1) lookup)
        String cached = redisTemplate.opsForValue().get("url:" + code);
        if (cached != null) {
            analyticsService.recordClickAsync(code, clientIp, userAgent);
            return cached;
        }

        // 2. Database fallback
        Url url = urlRepo.findByShortCode(code)
            .orElseThrow(() -> new ResourceNotFoundException("Short URL not found: " + code));

        if (url.isExpired()) {
            throw new UrlExpiredException("This URL has expired");
```

```
        }

        // Re-populate cache
        redisTemplate.opsForValue().set("url:" + code, url.getOriginalUrl(), Duration.ofHours(24
        analyticsService.recordClickAsync(code, clientIp, userAgent);
        return url.getOriginalUrl();
    }

    private String generateUniqueCode() {
        String chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
        Random random = new SecureRandom();
        String code;
        do {
            code = random.ints(0, chars.length())
                .limit(CODE_LENGTH)
                .mapToObj(i -> String.valueOf(chars.charAt(i)))
                .collect(Collectors.joining());
        } while (urlRepo.existsByShortCode(code));  // ensure uniqueness
        return code;
    }
}
```

10.4 Controller Layer

JAVA UrlController.java

```java
@RestController
@RequiredArgsConstructor
public class UrlController {

    private final UrlService urlService;
    private final RateLimiter rateLimiter;

    @PostMapping("/api/shorten")
    public ResponseEntity<UrlResponse> shorten(
            @RequestBody @Valid ShortenRequest request,
            @AuthenticationPrincipal UserDetails user,
            HttpServletRequest httpRequest) {

        String clientIp = httpRequest.getRemoteAddr();
        if (!rateLimiter.isAllowed(clientIp)) {
            return ResponseEntity.status(429).build();
        }

        Long userId = ((AppUserDetails) user).getId();
        return ResponseEntity.ok(urlService.shorten(request, userId));
    }

    @GetMapping("/{code}")
    public ResponseEntity<Void> redirect(
            @PathVariable String code,
            HttpServletRequest request) {

        String targetUrl = urlService.resolve(
            code,
            request.getRemoteAddr(),
            request.getHeader("User-Agent")
        );

        return ResponseEntity
            .status(HttpStatus.FOUND)  // 302 redirect
            .location(URI.create(targetUrl))
            .build();
    }

    @GetMapping("/api/urls/{code}/stats")
    @PreAuthorize("isAuthenticated()")
    public ResponseEntity<UrlStats> getStats(@PathVariable String code) {
        return ResponseEntity.ok(urlService.getStats(code));
```

```
    }
}
```

10.5 Repository Layer

JAVA UrlRepository.java

```java
public interface UrlRepository extends JpaRepository<Url, Long> {

    Optional<Url> findByShortCode(String code);

    boolean existsByShortCode(String code);

    List<Url> findByUserIdOrderByCreatedAtDesc(Long userId, Pageable pageable);

    @Query("SELECT u FROM Url u WHERE u.expiresAt < :now")
    List<Url> findExpiredUrls(@Param("now") LocalDateTime now);

    @Modifying
    @Query("UPDATE Url u SET u.clickCount = u.clickCount + 1 WHERE u.shortCode = :code")
    void incrementClickCount(@Param("code") String code);
}
```

10.6 Async Analytics

JAVA AnalyticsService.java

```java
@Service
@RequiredArgsConstructor
public class AnalyticsService {

    private final UrlRepository urlRepo;
    private final ClickRepository clickRepo;

    @Async  // Runs in separate thread pool — doesn't block redirect response
    public void recordClickAsync(String code, String ip, String userAgent) {
        urlRepo.incrementClickCount(code);

        Click click = new Click();
        click.setShortCode(code);
        click.setIpAddress(anonymizeIp(ip));  // GDPR: anonymize last octet
        click.setUserAgent(userAgent);
        click.setClickedAt(LocalDateTime.now());
        clickRepo.save(click);
    }

    private String anonymizeIp(String ip) {
        if (ip == null) return null;
        int lastDot = ip.lastIndexOf('.');
        return lastDot > 0 ? ip.substring(0, lastDot) + ".0" : ip;
    }
}
```

10.7 Scheduled Cleanup Job

JAVA CleanupScheduler.java

```java
@Component
@RequiredArgsConstructor
@Slf4j
public class CleanupScheduler {

    private final UrlRepository urlRepo;
    private final RedisTemplate<String, String> redisTemplate;

    // Run every day at 2 AM
    @Scheduled(cron = "0 0 2 * * *")
    public void cleanupExpiredUrls() {
        List<Url> expired = urlRepo.findExpiredUrls(LocalDateTime.now());
        log.info("Cleaning up {} expired URLs", expired.size());

        expired.forEach(url -> {
            redisTemplate.delete("url:" + url.getShortCode());
            urlRepo.delete(url);
        });

        log.info("Cleanup complete");
    }
}
```

10.8 Production Configuration

YAML application-prod.yml

```yaml
server:
  port: 8080
  compression:
    enabled: true
  http2:
    enabled: true

spring:
  datasource:
    url: jdbc:postgresql://\${DB_HOST}:5432/\${DB_NAME}
    username: \${DB_USER}
    password: \${DB_PASSWORD}
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 30000
      max-lifetime: 1800000

  data:
    redis:
      host: \${REDIS_HOST}
      port: 6379
      password: \${REDIS_PASSWORD}
      timeout: 2000ms

  jpa:
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
        default_batch_fetch_size: 25
    show-sql: false

jwt:
  secret: \${JWT_SECRET}
  expiration: 86400000

management:
  endpoints:
    web:
      exposure:
        include: health,metrics,prometheus
```

🚀 **FURTHER PRODUCTION ENHANCEMENTS**

CDN Integration: Put CloudFront/Cloudflare in front for edge caching of redirects — latency drops from ~20ms to <5ms globally. **Custom Domains:** Allow users to use their own domain (e.g., `go.mycompany.com/abc123`). **QR Code Generation:** On-demand QR code generation per short URL. **A/B Testing:** Route a percentage of traffic to alternate destinations. **Webhook Support:** Notify a URL on each click event. **OpenAPI Docs:** Springdoc-OpenAPI 3.x for automatic Swagger UI.

Quick Reference — Production Checklist

Category	Item	Implementation
Security	Authentication	JWT with refresh tokens, BCrypt cost 12
Security	Rate Limiting	Redis sliding window, 100 req/min per IP
Security	CORS	Whitelist specific origins
Performance	Caching	Redis with TTL, Cache-Aside pattern
Performance	Connection Pool	HikariCP, pool size = (cores * 2) + 1
Performance	N+1 Prevention	JOIN FETCH / @EntityGraph / batch size
Reliability	Health Checks	Actuator /health with DB + Redis checks
Reliability	Circuit Breaker	Resilience4j on external service calls
Observability	Metrics	Micrometer + Prometheus + Grafana
Observability	Tracing	Micrometer Tracing + Zipkin/Jaeger
Data	Migrations	Flyway for versioned schema migrations
Data	Backups	Daily snapshots + point-in-time recovery